混合专家模型（MoE）

Mixture of Experts (MoE)

中文修订版 · 图表与公式随文排版

原作者：@techNmak · Understanding AI 系列 · https://x.com/techNmak

本版逐段翻译正文、标题、图注和表格，在对应位置保留原始图片与数学公式；图中自然语言标签译为中文。行内公式和独立公式沿用原稿矢量内容，不改写符号、上下标或数值。代码标识符及可执行字符串保留。

中文使用霞鹜文楷（LXGW WenKai），英文和数字使用 Times New Roman。术语括注英文原词。原稿资料与历史陈述按作者原文翻译，不代表重新核查外部资料的最新状态。

专业术语翻译对照表

英文原词	中文译名	缩写／符号
Mixture of Experts	混合专家	MoE
Routing	路由	
Models	模型	
Mixture-of-Experts	混合专家	MoE
Experts	专家	
Token	词元	
Feed-forward	前馈	
Transformer	Transformer 模型	Transformer
Expert	专家	
Router	路由器	
Active Parameters	激活参数	
Total Parameters	总参数	
Wall-clock Time	实际耗时	
FLOPs	浮点运算次数	
Expert Capacity	专家容量	
Token Dropping	词元丢弃	
Dispatch	分派	
Core	核心	
Matrix Multiplication	矩阵乘法	
Feed-forward Network	前馈网络	FFN
Transformer Block	Transformer 块	
Attention	注意力机制	Attention
Inference	推理	
Training	训练	
Softmax	Softmax 归一化	Softmax
Expert Parallelism	专家并行	
Model	模型	

英文原词	中文译名	缩写／符号
Scaling	缩放	
Hidden Representation	隐藏表示	
Tokens	词元	
Sparse Activation	稀疏激活	
Latency	延迟	
Self-attention	自注意力	
Baseline	基线	
Matrix	矩阵	
Learned Projection	学习得到的投影	
SwiGLU	SwiGLU 门控激活	
Mental Model	理解框架	
Representation	表示	
Vector	向量	
Combine	合并	
Normalization	归一化	
Sparse Attention	稀疏注意力	
Dense Attention	稠密注意力	
Logit	逻辑值（归一化前的分数）	
Values	值	
Gradient	梯度	
Index	索引	
Renormalization	重新归一化	
Gating	门控	
Serving	推理服务	
Operator	算子	
Load Balancing	负载均衡	
Embeddings	嵌入向量	
Capacity Factor	容量因子	
Residual Path	残差路径	

英文原词	中文译名	缩写／符号
Residual Connection	残差连接	
Evidence	证据	
Auxiliary Loss	辅助损失	
Clipping	截断	
Load-balancing Loss	负载均衡损失	
Batching	批处理	
Kernel	内核	
Expert Specialization	专家专门化	
Shared Expert	共享专家	
Forward Pass	前向传播	
Memory Footprint	内存占用	
Precision	精度	
Quantization	量化	
Performance	性能	
Backward Pass	反向传播	
Arithmetic Intensity	算术强度	
Decode	解码	
All-to-all	全对全通信	
GEMM	通用矩阵乘法	GEMM
Memory Bandwidth	内存带宽	
Residency	驻留状态	
Encoder	编码器	
Decoder	解码器	
Per-group	逐组	
Resident	驻留	
Batch Size	批大小	
Activation	激活	
Query	查询	Q
Memory Traffic	内存流量	

目录导航

封面与说明 · 第 1 页
专业术语翻译对照表 · 第 2 页
混合专家模型（MoE） · 第 7 页
本手册将澄清哪些问题 · 第 8 页
为什么需要混合专家（Mixture of Experts）模型 · 第 9 页
专家（Expert）取代什么 · 第 10 页
一个方程，稀疏支持 · 第 11 页
MoE 位于区块中的位置 · 第 12 页
从词元（Token）向量（Vector）到专家（Expert）评分 · 第 14 页
选择、权重和艰难的决定 · 第 15 页
从 Logits 到两位选定的专家（Experts） · 第 16 页
结合专家（Expert）的输出 · 第 17 页
分配矩阵（Matrix）很重要 · 第 18 页
MoE 早于 Transformer · 第 20 页
2017 年稀疏门控公式 · 第 21 页
稀疏专家（Experts）如何进入大型变形金刚 · 第 22 页
路由（Routing）预算变化超过一个数字 · 第 23 页
静态执行约束，不是MoE的定义 · 第 25 页
容量因素实际上能买到什么 · 第 26 页
溢出行为是一种设计选择 · 第 27 页
为什么负载不平衡会自我强化 · 第 28 页
重要性和负荷不是同一个数量 · 第 29 页
统一专家（Expert）使用的可区分压力 · 第 30 页
稳定性与负载均衡是不同的问题 · 第 32 页
词元（Token）选择是常见的，不是强制性的 · 第 33 页
块稀疏内核改变了容量权衡 · 第 34 页
DeepSeekMoE改变了专家（Expert）库本身 · 第 35 页
DeepSeek-V3“无损”的实际含义是什么 · 第 36 页
最受虐的MoE对比 · 第 37 页
更多存储的专家容量（Expert Capacity），无需在玩具层中进行更多专家（Expert）工作 · 第 38 页
路由器（Router）两个阶段都存在，但问题不同 · 第 39 页
调度、计算、组合 · 第 40 页
为什么较少的活动参数仍然会运行得很差 · 第 41 页
专家（Experts）会成为“数学专家”和“代码专家”吗？ · 第 42 页
三种设计，三种不同的MoE选择 · 第 43 页
教学 Top-2 词元（Token）选择路由器（Router） · 第 44 页

看起来仍然合理的错误 · 第 45 页
创造错误思维模式的捷径 · 第 46 页
一条链中的专家混合 · 第 47 页
主要论文和当前实施来源 · 第 48 页

混合专家模型（MoE）

对应原稿第 1 页

理解混合专家（Mixture of Experts）模型

从稀疏路由（Routing）到现代 MoE 语言模型（Models）

面向人工智能工程师、开发人员和认真学习者的简短技术手册

@techNmak

手册07 - 理解AI系列

本手册将澄清哪些问题

对应原稿第 2 页

阅读导引

专家（Experts）混合层很容易概括为“将每个词元（Token）路由给几个专家”。这句话是正确的，但它隐藏了几乎所有使现代 MoE 模型（Models）变得有趣的工程和数学细节。

读完本手册后，您应该能够解释：

• 当Transformer 前馈（Feed-forward）层变成MoE 层时会发生什么变化？

• 路由器（Router）日志、得分、Top-k选择和专家（Expert）加权如何结合在一起；

• 为什么总参数（Total Parameters）、活动参数、FLOPs、权重内存和实际耗时（Wall-clock Time）是不同的数量？

• 为什么会出现负载不平衡，哪些辅助损耗实际上得到了优化，以及为什么路由器（Router）稳定性是一个单独的问题；

• 专家能力和词元掉落意味着什么，以及为什么两者都不是 MoE 的基础；

• 为什么分布式MoE需要词元（Token）调度、专家（Expert）并行通信和高效的分组专家计算；

• 在不改变核心MoE 理念的情况下，Mixtral、DeepSeekMoE/DeepSeek-V3 和Qwen3 等设计有何不同。

先备知识

您应该了解基本的 Transformer 模块、前馈网络（Feed-forward Network）的作用、矩阵乘法（Matrix Multiplication）、softmax 以及训练（Training）和推理（Inference）之间的区别。早期的《理解注意力机制（Attention）手册》是有用的上下文，但 MoE 路由（Routing）不是注意力机制。

适用范围

本手册重点介绍现代语言模型（Model） Transformer 中的稀疏专家（Expert）层。它涵盖了足够的专家并行（Expert Parallelism）性来解释执行成本，但完整的分布式系统设计属于单独的系统手册。

研究依据：[1]-[4]、[9]-[16]

为什么需要混合专家（Mixture of Experts）模型

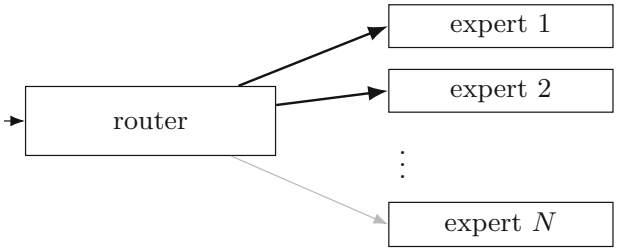
对应原稿第 3 页

1 - 条件计算

密集的神经网络层将相同的学习参数应用于到达它的每个词元（Token）。因此，增加层宽度或深度会增加参数容量和为每个词元执行的计算。

专家（Expert）混合引入了另一个缩放（Scaling）轴：条件计算。该模型（Model）可以拥有大量参数，同时仅激活特定输入的子集。中心目标不是“自由参数”。它在某种程度上将模型中存储的参数容量与应用于一个词元（Token）的专家计算量解耦。

对于词元（Token）表示 x,，稀疏专家（Experts）层从较大的存储体中选择一小组专家。如果有 N专家但仅选择了 $k \ll N$，则其他专家无需执行该词元。

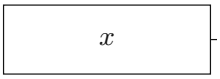


路由器（Router）

专家（Expert）1

专家（Expert）2

专家 N



稀疏选择取决于输入。两个相邻的词元（Tokens）可以使用不同的专家（Experts），并且相同的词元可以在不同层使用不同的专家，因为它的隐藏表示（Hidden Representation）通过网络发生变化。

重要。稀疏激活（Sparse Activation）减少了专家（Expert）计算的执行。它不会自动减少存储所有专家权重所需的内存，也不能保证较低的实际运行延迟（Latency）。

研究依据：[2]、[4]、[7]

专家（Expert）取代什么

对应原稿第 4 页

2 - 密集基线（Baseline）

在标准 Transformer 中，前馈（Feed-forward）子层独立应用于注意子层之后的每个词元（Token）。可以编写一个简化的两矩阵（Matrix）前馈网络（Feed-forward Network）

$$\mathrm{FFN}(x) = W_2\,\phi(W_1 x),$$

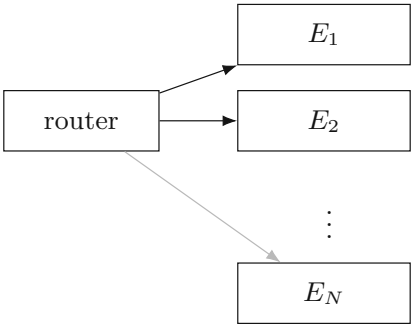
其中 $x \in \mathbb{R}^{d_{\text{model}}}$, W_1扩展隐藏维度， ϕ是非线性， W_2投影回模型（Model）维度。

现代 LLMs 通常使用门控变体，例如 SwiGLU，其中涉及额外的学习得到的投影（Learned Projection）。确切的 FFN 公式是特定于体系结构的。有用的基线（Baseline）更简单：密集的 Transformer 在该层具有一组 FFN 参数，并且每个词元（Token）都使用同一组参数。

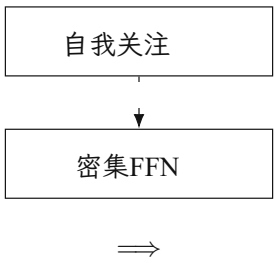
MoE Transformer 通常会更改此子层，而不是注意机制。该层拥有多个具有相同输入/输出形状但参数不同的 FFN，而不是一个 FFN：

$$E_1(x), E_2(x), \ldots, E_N(x).$$

路由器（Router）决定由哪一个处理每个词元（Token）。



路由器（Router）



理解框架（Mental Model）。在现代稀疏 Transformer MoE 中，“专家（Expert）”通常是层内的前馈（Feed-forward）子网络。它不是一个完全独立的语言模型（Model）。

研究依据：[3]、[4]、[9]、[11]

一个方程，稀疏支持

对应原稿第 5 页

3 - 萌部层

令 $E_i(x)$为专家（Expert） i，并令 $G_i(x)$为该专家的路由器（Router）权重。可以编写一个通用的专家混合层

$$\mathrm{MoE}(x) = \sum_{i=1}^{N} G_i(x) E_i(x) \Big|.$$

The equation itself does not require sparsity. The computational advantage appears when most entries of $G(x)$ are

方程本身不需要稀疏性。当$G(x)$ 的大多数条目都是时，计算优势就会显现出来

正好为零，所以只有一个小的支撑集

$$\mathcal{S}(x) = \{i : G_i(x) \neq 0\}$$

需要进行评估。

对于Top-k词元（Token）选择路由（Routing），普通条件下为 $|\mathcal{S}(x)| = k$。然后输出是

$$y = \sum_{i \in \mathcal{S}(x)} g_i(x) E_i(x).$$

专家（Experts）们必须产生兼容的输出形状，以便将他们的贡献结合起来。原则上，它们可以具有不同的内部架构，但大型语言模型（Models）通常使用同质的 FFN 专家，因为规律性简化了训练（Training）和执行。

这个公式也解释了 MoE 没有改变的内容。词元（Token）表示（Representation）在 MoE 边界处仍然具有模型（Model）的普通隐藏维度。稀疏性存在于参数选择和专家（Expert）执行中，而不是存在于较小的词元向量（Vector）中。

重要。相同的高级方程式涵盖许多不同的路由器。 Softmax Top-k,基于 sigmoid 的亲和力、平衡分配和专家（Expert）选择方法在 $G(x)$的构建方式上有所不同。该等式是抽象，而不是一种实现的规范。

研究依据：[2]、[7]、[9]、[10]、[11]

MoE 位于区块中的位置

对应原稿第 6 页

4 - 变压器放置

Transformer 块（Transformer Block）通常将词元（Token）混合操作（注意力机制（Attention））与位置前馈（Feed-forward）操作交替使用。稀疏变压器 MoE 模型（Models）通常用专家（Expert）库替换部分或全部密集 FFN 子层。

示意性的预归一化块可以被认为是

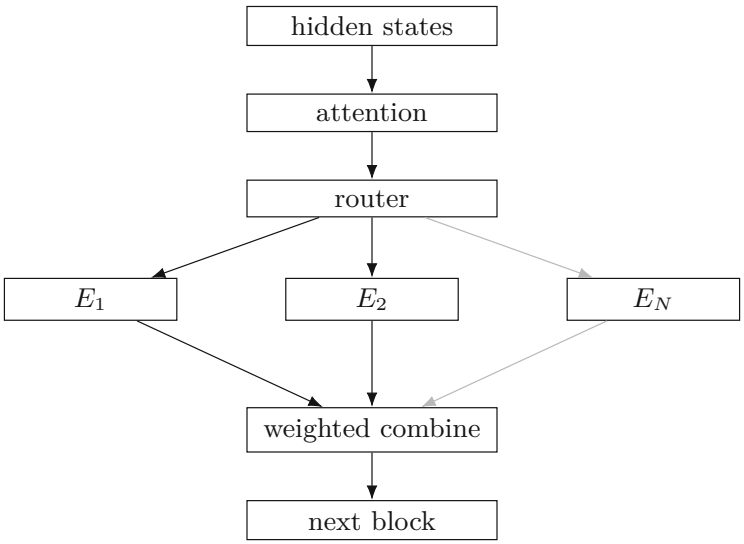
$$h' = h + \text{Attention}(\text{Norm}(h)),$$

$$h'' = h' + \text{FFN}(\text{Norm}(h')).$$

MoE 版本将第二个计算更改为

$$h'' = h' + \text{MoE}(\text{Norm}(h')).$$

确切的归一化（Normalization）和残差排序因模型（Model）而异，因此这些方程是结构草图而不是通用实现。



加权组合

隐藏状态

下一个街区

关注

路由器（Router）

GShard 将稀疏的 MoE 层放置在 Transformer 中，并在许多设备上扩展它们。 Switch Transformer 后来简化了将词元（Token）路由（Routing）给一位专家（Experts）的过程。 Mixtral 用稀疏专家层替换前馈（Feed-forward）块，并将每个词元路由到两个专家。

重要。 MoE 和注意力机制（Attention）稀疏性是不同的想法。模型（Model）可以具有稠密注意力（Dense Attention）和稀疏 FFN、稀疏注意力（Sparse Attention）和密集 FFN，或两者兼而有之。

研究依据：[3]、[4]、[9]

从词元（Token）向量（Vector）到专家（Expert）评分

对应原稿第 7 页

5 - 路由器（Router）

对于常见的线性路由器（Router），取一个词元（Token）表示 $x \in \mathbb{R}^d$和一个路由器矩阵（Matrix）

$$W_r \in \mathbb{R}^{d \times N}.$$

使用行向缩放因子示法，路由器（Router）逻辑是

$$h = xW_r \in \mathbb{R}^N.$$

为每位专家（Expert）生成一个 Logit。在 softmax 路由器（Router）中，

$$\frac{e^{h_i}}{\sum_{j=1}^{N} e^{h_j}}.$$

$$p_i(x) =$$

这些值在稀疏之前形成密集的概率分布。

该路由器（Router）相对于广泛的 FFN 专家（Experts）来说很小，但它不是免费的。其分数计算随着 $d \times N$.Switch Transformer 的增长而增长，Switch Transformer 明确指出了该路由器术语，同时观察到，当仅选择一名专家时，专家计算不需要随着专家总数的增长而增长。

有两个术语细节很重要。

• 路由器（Router）logit：路由（Routing）网络产生的预归一化（Normalization）分数。

• 门/亲和权重：在应用路由器（Router）选择的转换后用于对专家（Experts）进行排名和/或加权的数量。

Softmax 并不总是将这两者联系起来。例如，DeepSeek-V3 使用 sigmoid 计算路由专家（Expert）亲和力，并对所选组合的亲和力进行标准化。

理解框架（Mental Model）。路由器（Router）不“理解专家（Expert）名字”。它学习从当前隐藏表示（Hidden Representation）到数值分数的映射，其训练（Training）压力来自任务损失，并且根据设计，平衡/稳定性目标。

研究依据：[2]、[4]、[11]

选择、权重和艰难的决定

对应原稿第 8 页

6 - TOP-K 路由（Routing）

在计算完所有专家（Experts）的分数后，词元（Token）选择路由（Routing）仅保留一小部分。对于softmax Top-k路由器（Router），定义

$$\mathcal{S}(x) = \text{TopK}(p(x), k).$$

通用输出是

$$y = \sum_{i \in \mathcal{S}(x)} \tilde{p}_i(x) E_i(x).$$

$\tilde{p}_i$?到底是什么？没有单一的通用答案。

• Shazeer 的稀疏门在未选择的 logits 设置为 $-\infty$.后应用 softmax

• Switch 使用选定的路由器（Router）概率作为其Top-1 专家（Expert）的门值。

• Mixtral 描述了 Top-Klogits 上的 softmax。

• DeepSeek-V3 使用 sigmoid 关联性，使用 Top-K,进行选择，然后标准化所选关联性。

这就是为什么手册不应该写“Top-k总是重新规范化原始 softmax 概率”作为定义。

如何训练艰难的选择?

索引（Index）选择本身是离散的。然而，在所选集合不改变的区域内，所选门值是路由器（Router）分数的可微函数。沙泽尔等人。当 $k > 1$,和 Switch 指出其门值即使使用 Top-1 路由（Routing）也提供可微分路径时，明确记录所选专家（Experts）的非零导数。辅助路由器损耗可以提供额外的梯度（Gradient）信号。

在选择边界处，映射是不连续的。在实践中，成功的 MoE 训练（Training）不需要假装 Top-k索引（Index）操作在任何地方都是平滑的函数。

研究依据：[2]、[4]、[9]、[11]

从 **Logits** 到两位选定的专家（**Experts**）

对应原稿第 9 页

7 - 工作路线 I

考虑四位专家（Experts）和一个带有路由器（Router） logits 的词元（Token）

$$h = [2.0,\ 1.2,\ 0.1,\ -0.4].$$

应用 softmax 可以得到小数点后四位，

$$p \approx [0.5919,\ 0.2659,\ 0.0885,\ 0.0537].$$

对于 Top-2 路由（Routing），选择专家（Experts） 1 和 2。

仅对于此有效示例，假设实现重新规范化所选概率，使它们总和为 1：

$$\tilde{p}_1 = \frac{0.5919}{0.5919 + 0.2659} \approx 0.6900,$$

$$\tilde{p}_2 = \frac{0.2659}{0.5919 + 0.2659} \approx 0.3100.$$

稀疏门变成

$$G(x) \approx [0.6900,\ 0.3100,\ 0,\ 0].$$

Expert	logit	dense softmax	selected weight
E_1	2.0	0.5919	0.6900
E_2	1.2	0.2659	0.3100
E_3	0.1	0.0885	0
E_4	-0.4	0.0537	0

专家（Expert）

对数

密集softmax

选定的重量

该算术显示了两种不同的操作：对专家（Experts）进行排名并确定每个选定专家的贡献程度。

重要。上面的重整化步骤是教学性的，而不是通用的 MoE 规则。在为真实架构重现此计算之前，请务必检查模型（Model）的路由器（Router）定义。

研究依据：[2]、[9]、[11]

结合专家（Expert）的输出

对应原稿第 10 页

8 - 工作路线 II

使用选定的权重继续前面的示例

$$\tilde{p}_1 \approx 0.6900, \qquad \tilde{p}_2 \approx 0.3100.$$

假设两位选定的专家（Experts）返回

$$E_1(x) = [1.0, -0.5], \qquad E_2(x) = [2.0, 1.5].$$

那么 MoE 输出是

$$y = 0.6900E_1(x) + 0.3100E_2(x),$$

so

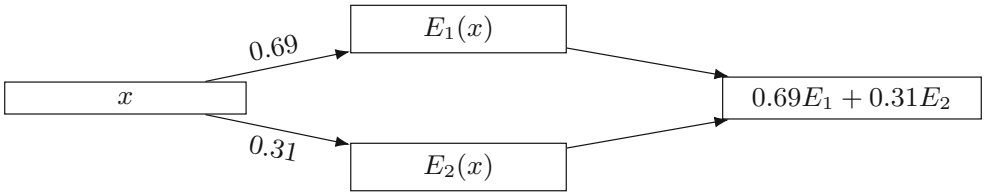
$$y \approx [1.3100,\ 0.1201].$$

第一个坐标是

$$0.6900(1.0) + 0.3100(2.0) = 1.3100,$$

第二个是

$$0.6900(-0.5) + 0.3100(1.5) \approx 0.1201.$$



这种加权组合就是为什么 Top-2 路由（Routing）不仅仅是“选择两名专家（Experts）并对他们进行平均”。路由器（Router）可以不均匀地调节它们的贡献。

在 Transformer 中，该专家（Expert）结果根据该模型（Model）的架构参与该块的普通残差/归一化（Normalization）结构。

理解框架（Mental Model）。路由（Routing）选择计算发生的位置。专家（Expert）函数计算候选变换。门权重决定如何组合选定的转换。

研究依据：[2]、[9]

分配矩阵（Matrix）很重要

对应原稿第 11 页

9 - 批量路由（Routing）

当我们将许多词元（Tokens）一起路由（Routing）时，MoE 的执行会成为系统问题。对于 T词元和 N专家（Experts）来说，二进制路由图可以表示为

$$R \in \{0,1\}^{T\times N},$$

其中 $R_{t,i}$= 1表示词元 t分配给expert i.

对于四个词元（Tokens）、四个专家（Experts）和 Top-2 路由（Routing），请考虑

$$\begin{bmatrix}1 & 1 & 0 & 0\end{bmatrix}$$

1 1 0 0

$$R = \begin{vmatrix}1 & 0 & 1 & 0\\ 0 & 1 & 1 & 0\\ 1 & 0 & 0 & 1\end{vmatrix}.$$

1 0 1 0

0 1 1 0

1 0 0 1

每行总和为 2，因为每个词元（Token）使用两个专家（Experts）。列总和给出专家负载：

$$[3,\ 2,\ 2,\ 1].$$

专家（Expert）作业总数为$4 \times 2 = 8$，但作业不平衡。

这个简单的矩阵（Matrix）揭示了几个后来的概念：

• 专家（Expert）负载：列中路由词元（Tokens）的数量；

• 容量：实施允许专家（Expert）处理多少任务；

• 调度：由需要词元（Token）行的专家（Experts）重新组合词元行；

• 组合：将处理后的专家（Expert）输出返回到原始词元（Token）顺序。

当专家（Experts）生活在不同的设备上时，该逻辑矩阵（Matrix）的列对应于可能需要移动的工作

通过网络。

重要。如果模型（Model）的分配矩阵（Matrix）产生很小的专家（Expert）批次、严重的不平衡或昂贵的通信，那么该模型可能在数学上稀疏，而在操作上效率低下。

研究依据：[2]、[8]、[16]

MoE 早于 Transformer

对应原稿第 12 页

10 - 历史

混合专家（Mixture-of-Experts）的想法比现代的 LLMs 更古老。 Jacobs、Jordan、Nowlan 和 Hinton 于 1991 年引入了本地专家（Experts）的自适应混合：单独的专家网络学习不同的训练（Training）案例子集，同时门控（Gating）机制在它们之间分配案例。

现代稀疏 LLM 谱系添加了两个关键成分。

稀疏条件计算

沙泽尔等人。 (2017) 将 MoE 转变为通用神经网络组件，具有潜在的数千个前馈（Feed-forward）专家（Experts）和可训练的稀疏门。只有少数专家对每项意见进行了评估。他们的语言模型（Models）将 MoE 放置在循环层之间而不是 Transformer 内部。

变压器秤 MoE

然后，GShard 将稀疏门控 MoE 集成到 Transformers 中，并演示了超过 600B 参数的多语言翻译模型（Model）的自动分片。 Switch Transformer 简化了对单个选定专家（Expert）的路由（Routing），并研究了大规模的训练（Training）稳定性、负载平衡和容量。

年份	工作	持久的想法
1991	Jacobs et al.	学习当地专家的门控
2017	沙泽尔等人。	sparse Top-k conditional computation
2020	GShard	超大规模分布式变压器 MoE
2022	开关	Top-1 路由和简化的稀疏执行
2024+	现代 LLM MoE	新的专家粒度、平衡和推理服务设计

雅各布斯等人。

GShard

稀疏 Top-k 条件计算

理解框架（Mental Model）。 MoE并不是当今巨头LLMs发明的技巧。现代模型（Models）是从学习模块化到分布式规模稀疏条件计算的漫长演变中的一个阶段。

研究依据：[1]-[4]

2017 年稀疏门控公式

对应原稿第 13 页

11 - 喧闹的 TOP-K

沙泽尔等人。提出了一个嘈杂的 Top-k门。在他们的符号中，密集的预选通值是

$$H_i(x) = (xW_g)_i + \epsilon_i \ \text{Softplus}((xW_{noise})_i),$$

具有高斯噪声 ϵ_i.的 Top-k运算符保留最大的 k值并将其余值设置为 $-\infty$,，然后 softmax 生成稀疏门：

$$G(x) = \text{Softmax}(\text{KeepTopK}(H(x), k)).$$

因此，未选定的专家（Experts）收到的门权重为零，并且不需要执行该输入。

噪声在原始设计中用于训练（Training）目的，包括支持负载平衡和探索。不应将其概括为“所有 MoE 路由器都会增加噪音”。许多后来的路由器使用不同的机制。

同一篇论文强调了仍然相关的两个系统成本：

• 缩小专家（Experts）批次：专家批大小为 b, N，每个词元（Token）分配为 k，平均专家在分布式聚合之前看到的词元分配大致为 kb/N;

• 网络带宽：如果专家（Experts）是分布式的，则词元（Token）表示和专家输出必须在持有这些专家的设备之间移动。

这些不是偶然的实施错误。它们直接遵循稀疏的、依赖于输入的路由（Routing）。

重要。最初的 2017 年门是基础性的，但现代路由器应该根据它们自己的定义来描述，而不是假设为嘈杂的 Top-k副本。

研究依据：[2]

稀疏专家（Experts）如何进入大型变形金刚

对应原稿第 14 页

12 - GShard 和开关

GShard 在 Transformer 中使用稀疏 MoE 并将其与自动分片相结合。其报告的系统在 2048 个 TPU v3 加速器上扩展了超过 600B 参数的多语言翻译模型（Model）。持久的架构点比标题数字更重要：专家（Expert）参数可以跨设备分布，同时仅为每个词元（Token）执行选定的专家路径。

Switch Transformer 进一步简化了路由（Routing）算法。每个词元（Token）都被发送给具有最高路由器（Router）概率的单个专家（Expert），而不是 Top-2。作者报告了三个实际动机：减少路由工作、减少专家能力要求以及降低通信复杂性。

对于 Top-1 交换机路由（Routing），

$$y = p_{i^*}(x) E_{i^*}(x), \qquad i^* = \arg\max_i p_i(x).$$

i^*= arg 最大值

所选择的概率仍然乘以专家（Expert）输出，因此即使 i^*的身份是一个艰难的选择，路由器（Router）也具有可微分的门值路径。

Switch 还使专家容量（Expert Capacity）和词元（Token）溢出成为核心工程概念，因为其面向 TPU 的实现使用静态限制的专家（Expert）批次。

重要。“Top-1 更简单”与“Top-1 普遍更好”并不是同一说法。后来的成功模型（Models）使用 Top-2、Top-8 和其他路由（Routing）预算。 k是一种架构和系统选择。

研究依据：[3]、[4]

路由（Routing）预算变化超过一个数字

对应原稿第 15 页

13 - TOP-1、TOP-2、TOP-K

所选专家（Expert）的数量 k通过多种方式影响专家路径。

Routing	Per-token expert work	Typical consequence
Top-1	one expert path	simpler dispatch, smallest expert-work budget
Top-2	two expert paths	mixture of two transformations, more assignments
Top-k	k expert paths	higher expert work/communication as k grows

路由（Routing）

前1名

前2名

顶部-k

每个词元（Token）的专家（Expert）工作

一条专家（Expert）路径

两条专家（Expert）路径

k 专家路径

典型后果

调度更简单，专家（Expert）工作预算最小

两种转换的混合，更多的作业

高级专家（Expert）工作/交流如 k

成长

如果专家（Experts）宽度保持固定，则从 Top-1 移动到 Top-2 大约会使该 MoE 层的 Expert-FFN 计算增加一倍。它不一定会使整个模型（Model）的 FLOP 翻倍，因为注意力机制（Attention）、嵌入向量（Embeddings）、标准化、共享专家和其他层仍然存在。

同样，如果专家（Expert）总数 N增长，而 k和专家形状保持固定，则存储的专家参数总数可以大幅增长，而不会按比例增加一个词元（Token）的专家工作量。但路由器（Router）得分随着N,的增长而增长，更大的权重库仍然需要存储/分片，并且分布式执行可能会变得更加困难。

现代示例使多样性具体化：Switch 使用 Top-1，Mixtral 8x7B 使用 Top-2，Qwen3-30B-A3B 报告 128 个专家（Experts），其中 8 个已激活，DeepSeek-V3 选择 8 个路由专家及其共享专家路径。

理解框架（Mental Model）。 N控制专家（Expert）存储体的规模。 k控制词元（Token）使用该存储体的数量。这两个数字都无法单独告诉您延迟（Latency）、内存或模型（Model）质量。

研究依据：[4]、[9]、[11]、[14]

静态执行约束，不是MoE的定义

对应原稿第 16 页

14 - 专家能力

动态路由（Routing）可以将不均匀数量的词元（Tokens）发送给不同的专家（Experts）。因此，一些实现为每个专家提供了固定的处理能力。

对于 Top-1 Switch Transformer 实施，专家容量（Expert Capacity）定义为

$$C = \frac{T}{N} \times c_f$$

其中 T是路由（Routing）组中的词元（Tokens）数量， N是专家（Experts）数量，c_f是容量因子（Capacity Factor）。

当 c_f= 1时，专家（Expert）槽总数恰好等于此Top-1设置中的词元（Token）计数。高于一的因素会为不均匀的路由（Routing）创建缓冲区，但会在静态形状的实现中引入填充、额外的内存和额外的工作。

这个公式经常被过于广泛地重复。对于一般的 Top-k路由（Routing），有 kT词元（Token）专家（Expert）分配，并且框架/论文在定义容量、分组、填充以及 k是否折叠到容量公式中的方式上有所不同。一些现代实施完全避免了固定的每位专家容量（Expert Capacity）。

重要。专家（Expert）能力是与特定路由（Routing）/运行时设计相关的执行策略。 MoE 在数学上需要稀疏的专家选择，而不是固定容量的缓冲区。

能力也是有范围的。它可以按批次、每个本地路由（Routing）组、每个专家（Expert）并行等级或其他单位来计算。该范围改变了分配器可见的不平衡情况。

研究依据：[4]、[7]、[16]

容量因素实际上能买到什么

对应原稿第 17 页

15 - 工作能力

采用 Top-1 交换机式路由（Routing）组

$$c_f = 1.25.$$

T= 1024 个词元,N= 16 个专家,

均匀平衡的负载将是

$$\frac{1024}{16}$$

= 64 词元（Tokens）/专家.

配置容量为

C= $64 \times 1.25 = 80$词元（Tokens）/专家.

对于所有专家（Experts）来说，运行时都有空间

$$16 \times 80 = 1280$$

专家（Expert）槽，尽管只存在 1024 个词元（Token）分配。 25% 的额外时段预算是防止失衡的缓冲。

假设专家（Expert） 3 收到 95 个词元（Tokens）。在固定容量的 Switch 行为下，只能容纳 80 个。剩余的

$$95 - 80 = 15$$

词元（Tokens）超出了该专家（Expert）的能力。在 Switch 中，溢出的词元会跳过专家计算并继续通过剩余路径。

增加 c_f会减少溢出的可能性，但会增加静态配置的专家（Expert）工作/内存量。减少它可以节省容量，但更容易溢出。

理解框架（Mental Model）。容量因子（Capacity Factor）是备用专家（Expert）槽和不平衡后果之间的系统权衡。它不是具有通用最佳值的质量旋钮。

重要。这个工作示例是 Top-1 和 Switch 特定的。不要对每个 Top-kMoE 实现盲目地重用 $T/N \times c_f$。

研究依据：[4]

溢出行为是一种设计选择

对应原稿第 18 页

16 - 词元掉落

较旧的固定容量 MoE 实现使词元丢弃（Token Dropping）变得可见，因为静态专家（Expert）缓冲区可能会溢出。在 Switch Transformer 中，溢出的词元（Token）不会接收稀疏专家计算，而是通过剩余连接向前传递。

该历史有时会变成错误的陈述“MoE 掉落词元（Tokens）”。现代证据（Evidence）表明为什么这是错误的。

巨型块

MegaBlocks 将专家（Experts）计算重新表述为块稀疏操作，可以有效地处理不平等的专家负载。它的设计明确避免了在丢弃词元（Tokens）和将所有专家填充到固定容量之间进行选择。

DeepSeek-V3

DeepSeek-V3 报告指出，其负载平衡足够有效，不会在训练（Training）期间丢弃词元（Tokens），并描述了旨在避免推理（Inference）期间丢弃词元的部署策略。

当前威震天核心

当前的 NVIDIA Megatron Core 文档将 moe_expert_capacity_factor=None 定义为默认情况下不丢弃词元。当静态形状或其他执行要求使容量限制模式有用时，容量限制模式仍然可用。

重要。“无滴”并不意味着“无成本”。可变的专家（Expert）负载仍然必须有效地表示、传达、调度和计算。

持久的区别是：

路由（Routing）不平衡= 强制词元（Token）丢失.

运行时可以通过动态缓冲区、稀疏内核、填充、容量限制或其他策略来处理不平衡。

研究依据：[4]、[8]、[11]、[16]

为什么负载不平衡会自我强化

对应原稿第 19 页

17 - 路由（Routing）崩溃

可训练的路由器（Router）可以自由地选择某些专家（Experts）而不是其他专家。如果没有反压力，这种偏好就会变得极端。

沙泽尔等人。描述了一种自我强化的故障模式：一些受青睐的专家（Experts）收到更多的例子，因此收到更多的任务梯度（Gradient）更新，改进得更快，并对路由器（Router）变得更有吸引力。其他专家可能缺乏培训。

负载不平衡会产生两个不同的问题。

优化问题

如果大多数数据流经一小部分专家（Experts），则该模型（Model）无法有效地使用其拥有的参数容量。未得到充分利用的专家从主要任务中获得的学习信号较少。

系统问题

当专家（Experts）使用不同的设备时，该步骤可能会受到负载最重的专家/等级的限制。一些

当其他设备处理大量词元（Token）时，设备会等待。容量有限的实现可能会溢出或填充。动态实现仍然面临落后者和不规则批大小的问题。

这两个问题解释了为什么负载平衡不仅仅是对相等直方图的表面偏好。

理解框架（Mental Model）。稀疏路由（Routing）在专家（Experts）学习和选择专家之间创建了一个反馈循环。平衡方法试图在不压倒主要语言模型（Model）目标的情况下，防止该循环崩溃到太少的专家身上。

重要。完全统一的路由（Routing）本身并不是学习目标。该模型（Model）仍然应该被允许在有用时以不同的方式使用专家（Experts）。面临的挑战是在保持有用的专业化的同时防止病理性失衡。

研究依据：[2]、[4]、[11]

重要性和负荷不是同一个数量

对应原稿第 20 页

18 - 早期平衡损失

2017 年稀疏门控的 MoE 论文分离了两个容易混淆的概念。

对于批次 X,，它将专家（Expert）重要性定义为门值的总和：

$$\text{Importance}(X) = \sum_{x \in X} G(x).$$

辅助损失（Auxiliary Loss）对这些专家（Expert）重要性值的变异系数进行了惩罚：

$$L_{\text{importance}} = w_{\text{importance}} \, \text{CV}(\text{Importance}(X))^2.$$

这里CV=标准差/的意思。

为什么还需要另一次损失？相同的总门权重并不能保证相同的词元（Token）计数。一位专家（Expert）可以

一个人收到一些权重较大的词元（Tokens），另一个人则收到许多权重较小的词元。即使总门质量看起来是平衡的，硬件也会看到不同数量的词元行。

沙泽尔等人。因此，还引入了一个面向负载的目标，与噪声门下的预期专家（Expert）分配相关。其确切的结构取决于嘈杂的 Top-k公式。

重要。这是一个有用的历史教训：“平衡路由器（Router）”没有得到明确说明。您必须询问方法是否平衡概率质量、硬分配计数、设备负载、序列局部负载或其他。

后来的工作简化或取代了这些目标。 Switch 使用涉及硬分配分数和平均路由器（Router）概率的单个可微代理。 DeepSeek-V3 使用动态路由（Routing）偏差作为主要的批次级平衡压力。

研究依据：[2]、[4]、[11]

统一专家（Expert）使用的可区分压力

对应原稿第 21 页

19 - 开关平衡损耗

对于 Top-1 路由（Routing），Switch Transformer 定义

$$\sum_{i=1}^{N} f_i P_i,$$

$L_{\text{aux}} = \alpha N$

哪里

$$f_i = \frac{1}{T} \sum_{x \in B}$$

$\mathbf{1}\{\arg\max p(x) = i\}$

是实际发送给 Expert i,的词元（Tokens）比例，

$$P_i = \frac{1}{T} \sum_{x \in B} p_i(x)$$

是分配给该专家（Expert）的平均路由器（Router）概率。

硬分数 f_i是不可微的，但 P_i是可微的。该产品将观察到的路由（Routing）负载耦合到可微的概率信号。

考虑四位专家（Experts）

$$f = [0.50, 0.25, 0.25, 0], \qquad P = [0.40, 0.30, 0.20, 0.10].$$

然后

$$N \sum_i f_i P_i = 4(0.20 + 0.075 + 0.05) = 1.30.$$

完美均匀 $f = P = [0.25, 0.25, 0.25, 0.25]$,

$$N \sum_i f_i P_i = 1.00.$$

因此，不平衡示例中的加权损失为1.30α，而均匀示例中的加权损失为 α。

重要。统一参考值不为零。当专家（Expert）计数发生变化时，因子 N使均匀情况的比例保持不变。

这个损失是为 Switch 的 Top-1 设置而设计的。在未检查该架构如何定义分配和平衡的情况下，请勿将该等式移植到另一个路由（Routing）方案中。

研究依据：[4]

稳定性与负载均衡是不同的问题

对应原稿第 22 页

20 - 路由器（Router） Z 丢失

MoE 路由器（Router）包含指数和硬路由（Routing）决策，因此大的路由器日志可能会产生数值和训练（Training）不稳定问题。 ST-MoE介绍了路由器z-loss

2

$$L_z(x) = \frac{1}{B}\sum_{i=1}^{B}\left(\log\sum_{j=1}^{N} e^{x_j^{(i)}}\right)^{:}$$

对于一批 B词元（Token）路由器（Router）-logit 向量超过 N专家（Experts）。

这种损失会惩罚大型对数标准化器，并鼓励路由器（Router）对数保持较小的幅度。 ST-MoE 报告称，这提高了实验中的训练（Training）稳定性，而不会因更严格的更新裁剪而出现质量损失。

该工作的总训练（Training）目标包括主要的交叉熵损失、负载平衡项以及具有单独系数的路由器（Router） z 损失：

$L_{\text{total}} = L_{\text{CE}} + c_B L_B + c_z L_z.$

这种区别很重要：

• 负载平衡损失决定了词元（Tokens）如何在专家（Experts）之间分配；

• 路由器（Router）z-loss 限制router-logit 规模以获得稳定性。

它们可以共存，因为它们解决不同的问题。

重要。将 z 损失称为“另一种负载平衡损失”就抹杀了它的目的。路由器（Router）可能在数值上表现良好但仍然不平衡，或者平衡但仍遭受不稳定的 logit 幅度。

研究依据：[7]

词元（Token）选择是常见的，不是强制性的

对应原稿第 23 页

21 - 其他路由（Routing）公式

Top-k词元（Token）选择路由（Routing）询问每个词元它想要哪些专家（Experts）。其他工作改变了分配问题本身。

基础层

BASE 将词元（Tokens）到专家（Expert）的分配制定为平衡线性分配问题。其设计强制每个专家拥有相同数量的词元，因此避免了该公式中的辅助平衡损失。

专家（Expert）选择

专家（Experts）选择反转了方向：每个专家选择得分最高的词元（Tokens），最多达到固定的桶大小。这通过构建固定了每个专家的容量，而词元可以由可变数量的专家选择。

方法	谁选择？	加载属性
Token-choice Top-k	token chooses k experts	expert loads can vary
基地	全局平衡分配	施工均等分配
专家选择	专家评选顶级词元	固定专家桶，可变专家/-词元

词元（Token）选择TOP-k

词元（Token）选择 k 专家

专家（Expert）负载可能会有所不同

这些方法说明了一个更广泛的观点：稀疏 MoE 是一系列条件计算架构，而不是一个不可变的路由器（Router）算法。

重要。路由（Routing）制定改变了语义、优化和执行。完美平衡负载的替代方案可能会对词元（Token）覆盖范围、因果关系、批处理（Batching）或实现施加不同的约束。“平衡”不足以宣告一款路由器（Router）具有普遍优越性。

研究依据：[5]、[6]

块稀疏内核改变了容量权衡

对应原稿第 24 页

22 - 无滴执行

动态的 MoE 自然会为不同的专家（Experts）提供不同的词元（Token）计数。密集加速器内核更喜欢规则的、足够大的矩阵。历史上，这种不匹配将运行时推向固定容量、填充或词元丢弃（Token Dropping）。

MegaBlocks 攻击内核（Kernel）级别的不匹配问题。它代表具有块稀疏矩阵（Matrix）运算的专家（Experts） FFN 计算，可以有效地处理可变专家负载。 MLSys 论文的中央系统声称，这消除了在丢弃溢出词元（Tokens）和将专家填充到共同容量之间的强制选择。

由此产生的理解框架（Mental Model）与 Switch 的静态缓冲区不同：



词元（Token）数量不等

仍然不平等

无需对 MoE 进行数学更改。变化在于稀疏、不平衡的专家（Expert）批次映射到 GPU 计算的方式。

当前的生产框架暴露了这两种行为。威震天核心支持容量因子（Capacity Factor）和词元丢弃（Token Dropping）策略，但其默认容量因子目前为“无”，记录为无词元丢弃。

理解框架（Mental Model）。 MoE算法设计和MoE内核（Kernel）设计是耦合的。如果由此产生的稀疏工作负载与硬件的映射效果不佳，那么路由器（Router）在数学上可能很有吸引力，但价格却很昂贵。

研究依据：[8]、[16]

DeepSeekMoE改变了专家（Expert）库本身

对应原稿第 25 页

23 - 细粒度和共享的专家（Experts）

DeepSeekMoE 提出了两项旨在专家专业化的架构变更。

细粒度专家（Expert）细分

从 K激活的 N专家（Experts）开始，本文在概念上将专家更精细地拆分为 mN更小的专家并激活 $mK.$，因为每个更小的专家包含更少的参数，所以设计可以在控制专家参数和计算预算的同时暴露更多的组合。

纯粹的组合说明显示了布线空间扩展的原因。从 8 位专家（Experts）中选择 2 位给出

$$\binom{8}{2} = 28$$

2

无序专家（Experts）集。分成 16 个较小的专家并选择 4 个给出

$$\binom{16}{4} = 1820.$$

4

这并不意味着“智力提高 65 倍”。它仅说明了更大的一组可能的专家（Expert）组合。

共享专家（Shared Expert）隔离

DeepSeekMoE 还保留了一些专家（Experts）作为广泛应用的共享专家，而路由专家仍然稀疏。其动机是让共享路径捕获共同知识并减少路由专家之间的冗余压力。

DeepSeek-V3 使用该系列设计：共享专家（Shared Expert）计算与选定的路由专家（Expert）输出一起添加。

重要。共享专家（Shared Expert）改变了主动计算会计。词元（Token）可以执行其路由专家（Experts）和共享专家路径，因此“Top-8”不一定是完整的 FFN 计算故事。

研究依据：[10]、[11]

DeepSeek-V3“无损”的实际含义是什么

对应原稿第 26 页

24 - 辅助无损耗平衡

DeepSeek-V3 使用 sigmoid 词元（Token）到专家（Experts）的关联性 $s_{i,t}$来为其路由（Routing）专家。在其主要的负载均衡机制中，每个专家都有一个路由偏差 b_i.Top-K选择使用

$$s_{i,t} + b_i,$$

但专家（Expert）的输出权重来自原始亲和力 $s_{i,t},$，而不是有偏差的分数。

在训练（Training）步骤结束时，过载专家（Experts）的偏差会减小，而负载不足的专家的偏差会增加。因此，偏差会改变谁被选中，而不会直接改变乘以专家输出的亲和力值。

这就是本文将其主要批处理级方法称为无辅助损失（Auxiliary Loss）的含义：通过动态路由（Routing）偏差施加负载压力，而不是通过与语言模型（Model）目标竞争的传统批处理级辅助损失项来施加负载压力。

有一个重要的资格。 DeepSeek-V3 还指出，它使用具有极小的系数的互补序列平衡损失来防止单个序列内部的极端不平衡。

该报告还指出，DeepSeek-V3 在训练（Training）期间不会丢弃词元（Tokens），并描述了节点限制路由（Routing）以限制通信。

重要。“DeepSeek-V3 不使用平衡损失”过于宽泛。准确的说法是，其主要的批次级平衡策略是基于偏差且无辅助损失的，同时仍然存在少量互补的序列平衡损失。

研究依据：[11]

最受虐的MoE对比

对应原稿第 27 页

25 - 总参数与活动参数

MoE 模型（Models）通常用两个参数计数来描述。

总参数（Total Parameters）

所有模型（Model）权重都存储在密集层、注意力机制（Attention）、嵌入向量（Embeddings）、路由器、共享专家（Experts）和完整的 RouteExpert 库中。这些权重对于检查点大小、内存容量、分片、加载和通信都很重要。

每个词元（Token）的活动参数

根据架构的核算，其计算用于一个词元（Token）的前向传播（Forward Pass）的参数子集。在 MoE 层中，这包括选定的专家（Experts）路径，并且可能包括共享专家。模型（Model）的密集部分也保持活跃。

在 Mixtral 8x7B 中最容易看出差异。其论文报告了模型（Model）可访问的总共 47B 个参数以及每个词元（Token） 13B 个活动参数。该论文明确警告说，主动参数分析无法捕获权重内存成本或硬件利用率。

批次级别上出现了进一步的微妙之处。词元（Token） A 可以激活专家（Experts） 1 和 4，而词元 B 可以激活专家 2 和 7。在大批量中，可以使用许多或所有专家，即使每个单独的词元仅激活少数专家。

重要。活动参数不是模型（Model）的内存占用（Memory Footprint），也不是延迟（Latency）的通用代理。内核（Kernel）效率、重量移动、路由（Routing）、通信、批量形状、精度（Precision）和硬件都很重要。

理解框架（Mental Model）。“每个词元（Token）稀疏”并不意味着“推理服务（Serving）系统中可以缺少大多数模型（Model）权重”。

研究依据：[9]、[11]、[14]、[16]

更多存储的专家容量（Expert Capacity），无需在玩具层中进行更多专家（Expert）工作

对应原稿第 28 页

26 - 工作参数示例

使用故意简单的两个矩阵（Matrix） FFN，以便算术是透明的。忽略偏见。

密集的 FFN 与

$$d_{\text{model}} = 1024, \qquad d_{\text{ff}} = 4096$$

有

$$2 \times 1024 \times 4096 = 8{,}388{,}608$$

重量参数。

现在创建一个包含 8 个较小专家（Experts）的 MoE。每个专家有 $d_{\text{ff}} = 2048$，所以一个专家有

$$2 \times 1024 \times 2048 = 4{,}194{,}304$$

重量。完整的专家（Expert）库拥有

$$8 \times 4{,}194{,}304 = 33{,}554{,}432$$

专家（Expert）权重。

如果每个词元（Token）都使用 Top-2 路由（Routing），则所选的专家（Expert）路径会接触

$$2 \times 4{,}194{,}304 = 8{,}388{,}608$$

专家（Expert）权重，等于密集玩具 FFN 的两个矩阵（Matrix）乘法的权重计数。路由器（Router）矩阵仅添加

$$1024 \times 8 = 8192$$

该玩具设置中的重量。

因此，这个构造的示例具有 4 倍的存储专家（Expert）库参数，同时匹配每个词元（Token）的密集基线（Baseline）的理想化专家矩阵（Matrix）参数计数。

重要。这是算术说明，不是性能（Performance）保证。真实的 LLM FFN 可以使用门控三投影设计、共享专家（Experts）、不同宽度、填充、量化（Quantization）、通信和融合内核。相同的参数触摸计数并不意味着相同的运行时间。

研究依据：[2]、[4]、[9]

路由器（Router）两个阶段都存在，但问题不同

对应原稿第 29 页

27 - 训练（Training）与推理（Inference）

在训练（Training）期间，许多词元（Tokens）被一起路由，并且梯度（Gradient）流经选定的专家（Expert）计算和路由器（Router）权重。专家仅从在该步骤中使用它的词元接收主任务梯度。平衡和稳定性目标（如果存在）会添加特定于路由器的训练信号。

这会产生批次级动态：专家（Expert）利用率会影响每个专家学习的量，而当前路由器（Router）会影响每个专家看到的数据分布。

在推理（Inference）过程中，任务是不同的。没有反向传播（Backward Pass），仅训练（Training）辅助损失也没有优化，但模型（Model）仍然需要计算路由（Routing）决策、调度词元（Token）状态、执行选定的专家（Experts）并组合输出。

对于自回归解码（Decode），每个序列通常为每个解码步骤贡献一个新词元（Tokens）。如果不批量处理许多请求，单个专家（Expert）一次只能收到很少的词元，从而使专家 GEMM 变得很小且效率低下。大批量推理服务（Serving）可以提高算术强度（Arithmetic Intensity），但也使路由（Routing）/负载模式变得更加重要。

路由（Routing）也是特定于层的。生成的词元（Token）可以在早期层选择一组专家（Experts），并在稍后选择另一组专家，因为其隐藏表示（Hidden Representation）发生变化。

重要。“MoE 训练（Training）是稀疏的，因此推理（Inference）自动很快”并不是一个有效的推论。仅当推理服务（Serving）实现可以有效地利用生成的路由（Routing）模式时，稀疏算术才有用。

研究依据：[2]、[9]、[11]、[16]

调度、计算、组合

对应原稿第 30 页

28 - 专家并行（Expert Parallelism）

大型专家（Experts）库可能不适合 GPU，或者可能效率不高。专家并行（Expert Parallelism）性将专家跨设备划分。因此，路由器（Router）可以选择物理上远离词元（Token）当前设备的专家。

常见的执行模式是：

1.计算路由（Routing）索引和权重；

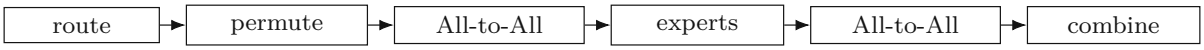
2. 由目的地专家（Expert）对词元（Token）状态进行排列/分组；

3. 将词元（Tokens）传送给托管这些专家（Experts）的设备；

4. 执行专家（Expert） FFN，通常作为分组或融合 GEMM；

5. 反馈专家（Expert）成果；

6. 按原始词元（Token）顺序取消排列和组合输出。



路线

排列

全部到全部

专家（Experts）

全部到全部

联合起来

当前的威震天核心记录了 All-to-All、AllGather 和灵活的词元（Token）分发后端。它的 All-to-All 调度程序明确地执行对专家（Expert）级别的调度、专家专业重排序，然后组合通信和重新排列。

重要。专家并行（Expert Parallelism）性不是 MoE 的定义。它是专家（Expert）库的分布式执行策略。小型 MoE 可以在一台设备上运行，无需 GPU 专家之间的通信。

研究依据：[3]、[11]、[16]

为什么较少的活动参数仍然会运行得很差

对应原稿第 31 页

29 - 性能（Performance）

稀疏专家（Expert）计算改变了性能（Performance）模型（Model）而不是删除它。多种成本相互竞争。

专家（Expert） GEMM 尺寸

对于许多专家（Experts），词元（Tokens）被分成较小的每个专家批次。微小或不规则的矩阵可能未得到充分利用

加速器。 2017 年的 MoE 论文将其称为收缩批次问题。

通讯

专家并行（Expert Parallelism）性将词元（Token）状态移至专家（Expert）设备并输出回来。如果计算/通信比率较差，则全对全通信（All-to-all）可以与专家计算相媲美。

体重运动

即使每个词元（Token）只使用少数专家（Experts），一批也可能涉及许多专家。因此，权重驻留和内存带宽（Memory Bandwidth）仍然很重要，尤其是在小批大小的情况下。

负载不平衡

如果工作没有有效重叠，最慢的专家（Expert）/等级可以确定暴露的步骤时间。

路由（Routing）和排列开销

分数计算、Top-k,元数据构建、排序/排列和组合操作都消耗时间。

现代系统通过分组 GEMM、块稀疏内核、拓扑感知路由（Routing）、融合调度和计算/通信重叠来响应。例如，DeepSeek-V3 明确地共同设计了跨节点通信的路由，并限制了词元（Token）可以联系的节点数量。

理解框架（Mental Model）。活动参数计数通过权重描述数学路径长度。性能（Performance）取决于硬件是否可以通过大型、本地、平衡良好的操作执行该路径。

研究依据：[2]、[8]、[9]、[11]、[16]

专家（Experts）会成为“数学专家”和“代码专家”吗？

对应原稿第 32 页

30 - 专家专业化

人们很容易将 MoE 变成一个人类故事：一位专家（Expert）学习数学，另一位专家学习代码，另一位专家学习法语。研究并没有证明这是一种普遍的理解框架（Mental Model）。

原始的稀疏门控 MoE 报告了其循环语言模型（Models）的专业化模式。但专业化在不同架构和层之间可能看起来有所不同。

Mixtral 对 The Pile 中的跨域进行了路由（Routing）分析。作者报告说，大多数数据集没有明显的领域级分配模式：ArXiv、PubMed、哲学和其他数据集具有类似的专家（Expert）分配分布。他们确实观察到更结构化的语法行为，例如缩进或某些词元（Token）模式的重复分配，以及某些层中的大量局部性。

ST-MoE 还发现专业化取决于上下文。它的分析报告了编码器（Encoder）专家（Experts）的专业化模式，而解码器（Decoder）专家没有表现出同样清晰的行为，并且多语言专业化并没有简化为简单的“每个专家一种语言”分区。

这使得三个概念值得区分：

• 路由（Routing）频率：选择哪些专家（Experts）；

• 功能专业化：专家（Experts）是否执行明显不同的转换；

• 人类可解释的专业化：这些差异是否清晰地映射到“数学”等标签。

重要。丰富多彩的专家（Expert）路由（Routing）图是有关选择模式的证据（Evidence），而不是自动证明专家存储了干净的语义域的证据。

研究依据：[2]、[7]、[9]

三种设计，三种不同的MoE选择

对应原稿第 33 页

31 - 现代型号

现代开放模型（Models）清楚地表明，“MoE”并不意味着一个专家（Expert）计数或路由（Routing）规则。

型号	总计/活跃	专家路由	选定的架构事实
混合 8x7B	47B / 13B	8位专家，Top-2	FFN 专家，softmax over Top-2 logits
DeepSeek-V3	671B / 37B	256 routed, Top-8	细粒度路由专家+共享专家，sigmoid亲和力
Qwen3-30B-A3B	30.5B / 3.3B	128位专家，Top-8	官方模型卡报告48层和8个激活专家

DeepSeek-V3

Qwen3-30B-A3B

256 条路线，前 8 名

这些标题参数计数来自每个模型（Model）的作者，应在该架构的会计中进行解释。它们是有用的描述，而不是直接可比较的延迟（Latency）测量。

Mixtral 的论文明确指出，主动参数分析不包括内存成本和硬件利用率。 DeepSeek-V3 将其专家（Experts）架构与节点限制路由（Routing）和自定义通信相结合。 Qwen3 的模型（Model）卡报告了比 Mixtral 大得多的专家库，每个词元（Token）有 8 位活跃专家。

重要。不存在“更多专家（Experts）”或“更少活跃参数”产生更快或更好模型（Model）的单调规则。专家规模、密集骨干网、路由（Routing）、精度（Precision）、并行性和实现都改变了比较。

研究依据：[9]、[11]、[14]

教学 Top-2 词元（Token）选择路由器（Router）

对应原稿第 34 页

32 - 透明实施

以下 PyTorch 风格的草图展示了概念阶段。这是原始的教学代码，而不是有效的

生产MoE执行。

```
# x: [tokens, d_model]
logits = x @ router_weight          # [tokens, experts]
probs = logits.softmax(dim=-1)

top_p, top_i = probs.topk(k=2, dim=-1)
top_p = top_p / top_p.sum(dim=-1, keepdim=True)

out = torch.zeros_like(x)
for slot in range(2):
    for expert_id, expert in enumerate(experts):
        mask = (top_i[:, slot] == expert_id)
        if mask.any():
            y = expert(x[mask])
            out[mask] += top_p[mask, slot, None] * y
```

它使数学步骤可见：

分数 →前2指数 →专家点评 →加权组合.

代码刻意隐藏的是大部分硬工程：容量策略、稳定的路由（Routing）精度（Precision）、

分布式调度、词元（Token）排列、分组 GEMM、通信重叠、共享专家（Experts）、负载平衡损失、训练（Training）噪声和专用内核。

大规模的嵌套 Python 循环将是灾难性的。生产系统将许多词元（Tokens）分组为同一专家（Expert），并调用批量/融合专家计算。

重要。此示例还重新规范化了所选的 softmax 概率。这是一种有效的教学惯例，而不是每个模型（Model）的通用路由器（Router）定义。

研究依据：[2]、[9]、[16]

看起来仍然合理的错误

对应原稿第 35 页

33 - 实施错误

1. 将 Top-k指数视为可微概率。

成员资格的决定是离散的。梯度根据路由器（Router）设计流经选定的权重和辅助目标，而不是通过假想的平滑 argmax。

2. 假设所有路由器都使用**softmax**。

DeepSeek-V3 对路由专家（Experts）使用 sigmoid 关联性。路由器（Router）标准化和加权必须与模型（Model）相匹配。

3. 对错误的集合进行标准化。

对所有专家（Experts）进行 Softmax 后进行掩码、在 Top-k掩码后进行 Softmax 以及仅在选定的 sigmoid 亲和力之间进行归一化（Normalization）是不同的计算。

4. 混淆词元（**Token**）计数和分配计数。

Top-k从 T词元（Tokens）创建最多 kT词元专家（Expert）分配。容量公式必须符合路由器（Router）/运行时约定。

5. 发货后原词元（**Token**）订单丢失。

专家（Expert）级执行需要足够的元数据来将输出组合回正确的词元（Token）行和路由（Routing）槽。

6. 在错误的范围内使用一位专家（**Expert**）的负载统计数据。

每个设备、每个组、每个序列和全局批次的平衡可能有所不同。

7. 假设活动参数等于常驻权重。

即使词元（Token）仅使用一个子集，推理服务（Serving）也可能需要整个分片专家（Expert）库。

8. 在不命名硬件和批次形状的情况下比较 **MoE** 速度。

稀疏执行效率对专家（Expert）批大小（Batch Size）、互连、精度（Precision）和内核高度敏感。

研究依据：[4]、[7]、[11]、[16]

创造错误思维模式的捷径

对应原稿第 36 页

34 - 常见的误解

“专家（Experts）是一个完整的 LLM。”在现代 Transformer MoE 中通常是错误的。专家通常是选定层内的 FFN/MLP 子网络。

“MoE 只运行一个微型模型（Model）。”词元（Token）激活稀疏专家（Expert）路径，但密集/共享参数仍然运行，并且完整的专家库仍然存在。

“总计 100B，活动 10B 意味着 10B 内存。”活动参数不是检查点或常驻权重内存。

“MoE 始终使用 Top-2。” Switch是Top-1，Mixtral Top-2，Qwen3和DeepSeek-V3使用更大的路由（Routing）预算。

“MoE 总是会掉落词元（Tokens）。”不。丢弃是一项容量策略。 MegaBlocks 和当前的无滴运行时间却显示出不同的情况。

“负载平衡意味着每个专家（Expert）都应该是相同的。”

平衡涉及利用率/负载。有用的功能差异仍然可能出现。

“每位专家（Expert）都学习一个领域。”路由（Routing）分析不支持将此作为通用解释。

“专家（Experts）越多，推理（Inference）速度就越快。”

更多的专家（Experts）会增加容量，但会恶化路线、体重移动和沟通。

“MoE 稀疏性就是稀疏的注意力。” MoE 稀疏参数/专家（Expert）激活（Activation）。稀疏注意力（Sparse Attention）改变了查询（Query）键的连接性。

研究依据：[4]、[8]、[9]、[11]、[14]

一条链中的专家混合

对应原稿第 37 页

35 - 完整的理解框架（Mental Model）

现代稀疏 Transformer MoE 可以被记住为九个相互关联的想法。

1. 从 FFN 开始。专家（Expert）库通常取代密集的前馈（Feed-forward）子层，而不是注意力机制（Attention）。

2. 得分。路由器（Router）将每个词元（Token）表示（Representation）映射到专家（Expert）分数或亲和力。

3. 选择。稀疏路由（Routing）规则选择一小部分专家（Experts）。

4. 计算。只有选定的路由专家（Experts）才能执行该词元（Token），以及架构定义的任何共享路径。

5. 合并（Combine）。路由器（Router）派生的权重将选定的专家（Expert）输出合并回一个词元（Token）表示（Representation）。

6.平衡。培训必须防止病态专家（Expert）使用不足/过度使用，同时又不能让平衡压力主导任务目标。

7. 稳定。路由器（Router）数值稳定性与负载平衡不同； z 损失等技术针对的是不同的故障模式。

8. 调度。在专家（Expert）并行系统中，词元（Token）状态通常通过全对全通信（All-to-all）转移到专家设备并输出返回。

9. 衡量正确的资源。总参数（Total Parameters）、活动参数、FLOP、内存容量、内存流量（Memory Traffic）、通信和实际耗时（Wall-clock Time）不可互换。

MoE 的持久承诺是条件计算：增加模型（Model）可用的参数容量，而不强制每个词元（Token）通过每个专家（Expert）参数。其持久的困难在于条件计算会产生动态的、不平衡的、分布式的工作，而密集的硬件无法很好地自动执行。

如果这两个陈述结合在一起，大多数 MoE 声明就会变得更容易评估。该架构既不是“用小模型（Model）换取巨大模型”，也不仅仅是“带有路由器（Router）的多个 FFN”。这是一个协同设计问题

涵盖优化、数值稳定性、稀疏计算、内存、通信和模型（Model）架构。

研究依据：[1]-[16]

主要论文和当前实施来源

对应原稿第 38 页

参考文献

1.R.A.雅各布斯、M.I.乔丹、S.J.诺兰、G.E.辛顿。当地专家（Experts）的适应性组合。神经计算 3(1):79-87, 1991。DOI: 10.1162/neco.1991.3.1.79。

2.N.Shazeer 等人。极其庞大的神经网络：稀疏门控专家混合层。 arXiv:1701.06538，2017。

3. D. Lepikhin 等人。 GShard：通过条件计算和自动分片扩展巨型模型（Models）。 arXiv:2006.16668，2020； ICLR 2021。

4. W. Fedus、B. Zoph、N. Shazeer。开关变压器：通过简单高效的稀疏性扩展到万亿参数模型（Models）。 JMLR 23(120):1-39, 2022。

5.M.刘易斯等人。基础层：简化大型稀疏模型（Models）的训练（Training）。 arXiv:2103.16716，2021。

6.Y.周等人。专家（Expert）混合与专家选择路由（Routing）。 NeurIPS 2022.arXiv:2202.09368.

7.B.佐夫等人。 ST-MoE：设计稳定且可转移的稀疏专家（Expert）模型（Models）。 arXiv:2202.08906，2022。

8. T. Gale、D. Narayanan、C. Young、M. Zaharia。 MegaBlocks：混合专家（Mixture-of-Experts）的高效稀疏训练（Training）。 MLSys 论文集，卷。 2023 年 5 月。arXiv:2211.15841.

9. A.Q.Jiang 等人。专家（Experts）的混合。 arXiv:2401.04088，2024。

10.D.戴等人。 DeepSeekMoE：迈向混合专家（Mixture-of-Experts）语言模型（Models）的终极专家专业化。 arXiv:2401.06066，2024。

11.DeepSeek-AI。 DeepSeek-V3 技术报告。 arXiv:2412.19437v2，2025；有关架构详细信息，请咨询官方 DeepSeek-V3 存储库/配置。

12. N. Muennighoff 等人。 OLMoE：开放专家混合语言模型（Models）。 arXiv:2409.02060，2024。

13.Qwen团队。 Qwen3 技术报告。 arXiv:2505.09388，2025。

14.Qwen团队。 Qwen3-30B-A3B官方型号卡。当前官方模型（Model）元数据参考了 2026 年 9 月的总/激活参数和专家（Expert）计数。

15. 米斯特拉尔人工智能。 Mixtral 8x7B 模型（Model）发布/源材料。仅在与主要 Mixtral 纸一致的情况下使用。

16. NVIDIA 威震天核心文档。 MoE 词元（Token）调度程序、专家并行（Expert Parallelism）性、路由器（Router）/容量配置。当前文档查阅于 2026 年 9 月。